บทที่ 9 กราฟ (Graph)

(ต่อ)

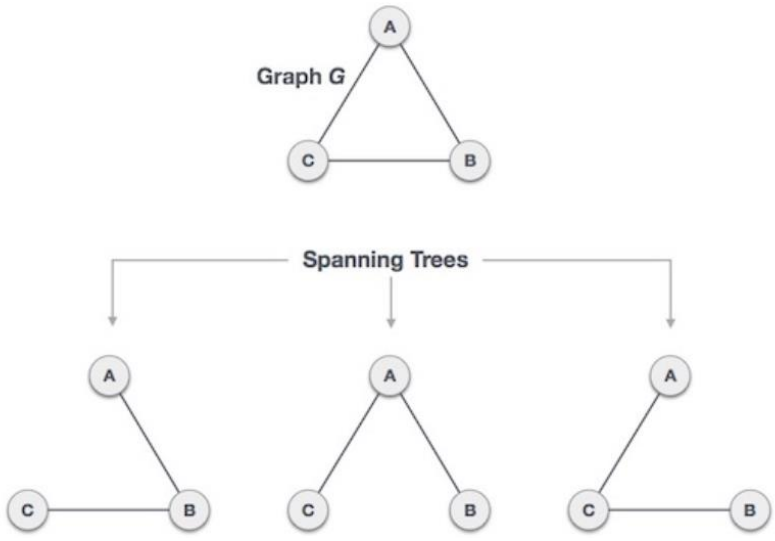
5 ต้นไม้ทอดข้ามที่เล็กที่สุด (Minimum spanning tree)

5.1 ต้นไม้ทอดข้าม/ต้นไม้แผ่ (Spanning tree)

กำหนดให้ G = (V, E) เป็นกราฟเชื่อมโยงที่ไม่ระบุทิศทาง (Undirected and Connected Graph) ต้นไม้ทอดข้าม (Spanning Tree) ของกราฟ G คือ ต้นไม้ที่

- เป็น Spans G ซึ่งหมายถึง กราฟที่ประกอบด้วยทุกจุดในกราฟ G (Spans G it includes every vertex of G) และ
- เป็นกราฟย่อยของ G ซึ่งกราฟย่อยนี้ประกอบด้วยเส้นที่ได้มาจากกราฟ G (A subgraph of G, every edge in the tree belongs to G)

<u>ตัวอย่าง</u> Spanning tree



คุณสมบัติของ Spanning tree (Spanning tree properties)

1. กราฟรูปหนึ่งมีได้หลาย Spanning tree
2. Spanning tree ทุกอันมีจำนวนจุด และจำนวนเส้นเท่ากัน

- ถ้า tree มี n จุด จะมีจำนวนเส้น n-1 เส้น

3. Spanning tree ไม่มี loop และไม่มี cycle
4. ถ้านำเส้นออกจาก spanning tee หนึ่งเส้น จะทำให้กราฟที่เหลือเป็น disconnected
    - กล่าวคือ spanning tree เป็น minimum connected graph นั่นเอง
5. การเพิ่มจุดขึ้นมาอีกหนึ่งจุดใน spanning tree ทำให้เกิด circuit หรือ loop
    - กล่าวคือ spanning tree เป็น maximally acyclic

5.2 ต้นไม้ทอดข้ามที่เล็กที่สุด (Minimum Spanning Tree: MST)

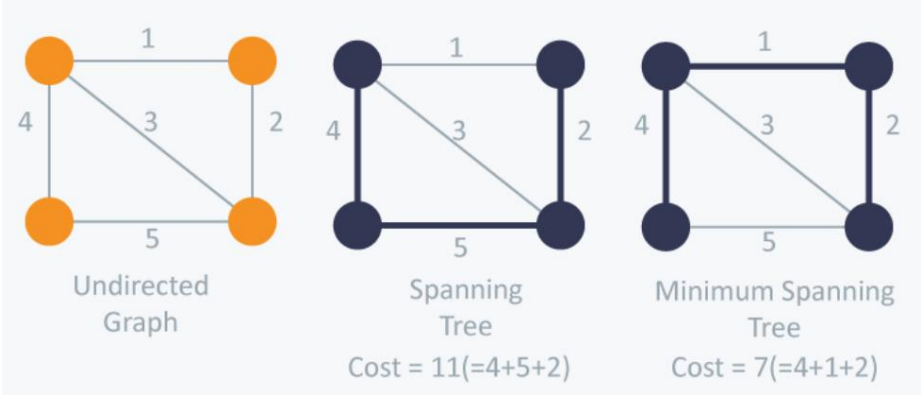
ในกราฟแบบมีน้ำหนัก (Weighted Graph) ต้นไม้ทอดข้ามที่น้อยที่สุด (Minimum Spanning Tree) คือต้นไม้ทอดข้ามที่มีน้ำหนักรวมของเส้นเชื่อม น้อยกว่าต้นไม้ทอดข้ามแบบอื่น ๆ ทั้งหมดในกราฟเดียวกัน

Weight อาจเป็น ระยะทาง ค่าโดยสาร ค่าปรับ หรือ traffic load เป็นต้น

วิธีการของ MST แบ่งได้เป็น 2 วิธี

- Kruskal’s algorithm
- Prim’s algorithm

<u>ตัวอย่าง</u> Spanning tree and Minimum Spanning tree

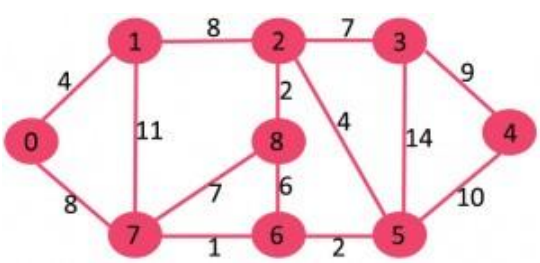


5.2.1 Kruskal's algorithm

วิธีการของ Kruskal's algorithm คือทำการเพิ่มเส้นเข้าไปใน spanning tree ทีละเส้น ตามการทำงานดังนี้

1. **เรียงลำดับเส้น (Edges) ทั้งหมด** ตามน้ำหนักจากน้อยไปมาก
2. **เลือกเส้นที่มีน้ำหนักน้อยที่สุดขึ้นมา** ตรวจสอบว่าเส้นนี้ทำให้เกิด "การวนซ้ำ" (Cycle) กับต้นไม้ทอดข้ามที่สร้างไว้ก่อนหน้าหรือไม่
   - หาก **ไม่เกิด** cycle ให้รับเส้นนี้เข้าไว้ในต้นไม้
   - หาก **เกิด** cycle ให้ตัดเส้นนี้ทิ้งไป
3. **ทำซ้ำขั้นตอนที่ 2** จนกระทั่งมีจำนวนเส้นในต้นไม้เท่ากับ **V-1** เส้น (เมื่อ V คือจำนวนจุดยอดทั้งหมด)

<u>ตัวอย่าง 5.1</u> หา Minimum spanning tree โดยใช้ Kruskal's algorithm



กราฟนี้ประกอบด้วย 9 จุด และจำนวนเส้น 14 เส้น ดังนั้น minimum spanning tree จะมี (9-1) = 8 เส้น

เรียงค่าน้ำหนักของแต่ละเส้นจากน้อยไปมาก

After sorting:

Weight	Src	Dest
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

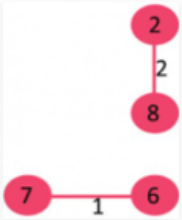
After sorting:

Weight	Src	Dest
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

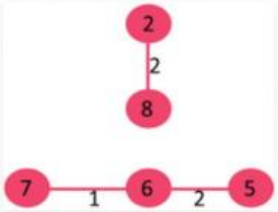
Now pick all edges one by one from sorted list of edges

1. *Pick edge 7-6:* No cycle is formed, include it.

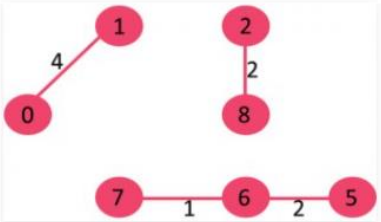
2. *Pick edge 8-2:* No cycle is formed, include it.



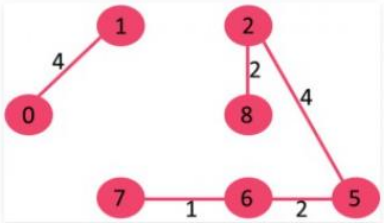
3. *Pick edge 6-5:* No cycle is formed, include it.



4. *Pick edge 0-1:* No cycle is formed, include it.



5. *Pick edge 2-5:* No cycle is formed, include it.

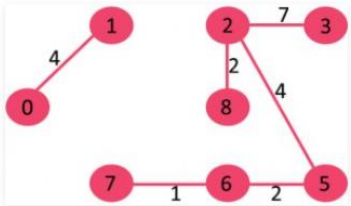


After sorting:

Weight	Src	Dest
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

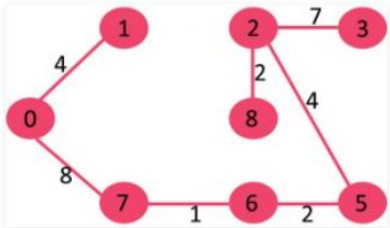
6. *Pick edge 8-6:* Since including this edge results in cycle, discard it.

7. *Pick edge 2-3:* No cycle is formed, include it.



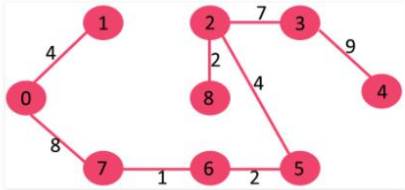
8. *Pick edge 7-8:* Since including this edge results in cycle, discard it.

9. *Pick edge 0-7:* No cycle is formed, include it.



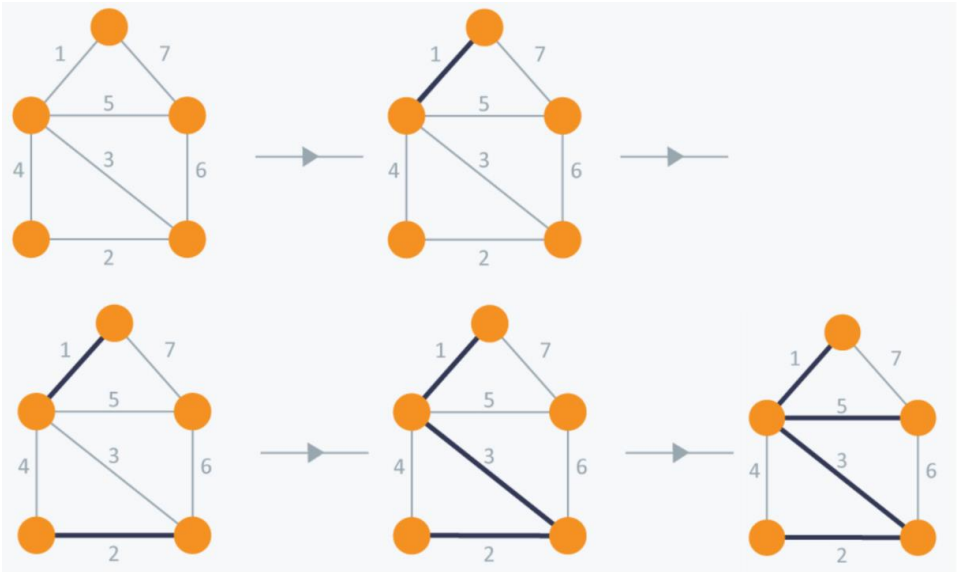
10. *Pick edge 1-2:* Since including this edge results in cycle, discard it.

11. *Pick edge 3-4:* No cycle is formed, include it.



Since the number of edges included equals (V − 1), the algorithm stops here.

<u>ตัวอย่าง 5.2</u> หา Minimum spanning tree โดยใช้ Kruskal’s algorithm

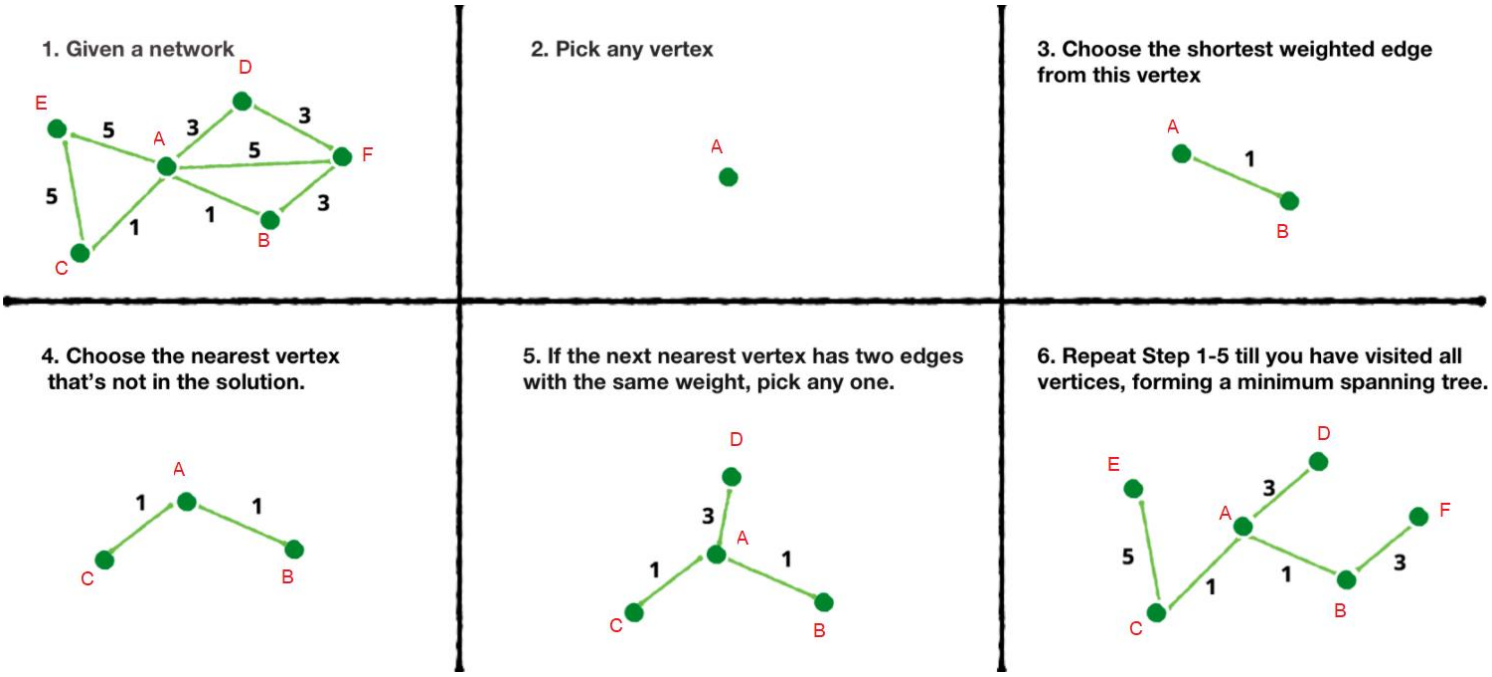


5.2.2 Prim’s algorithm

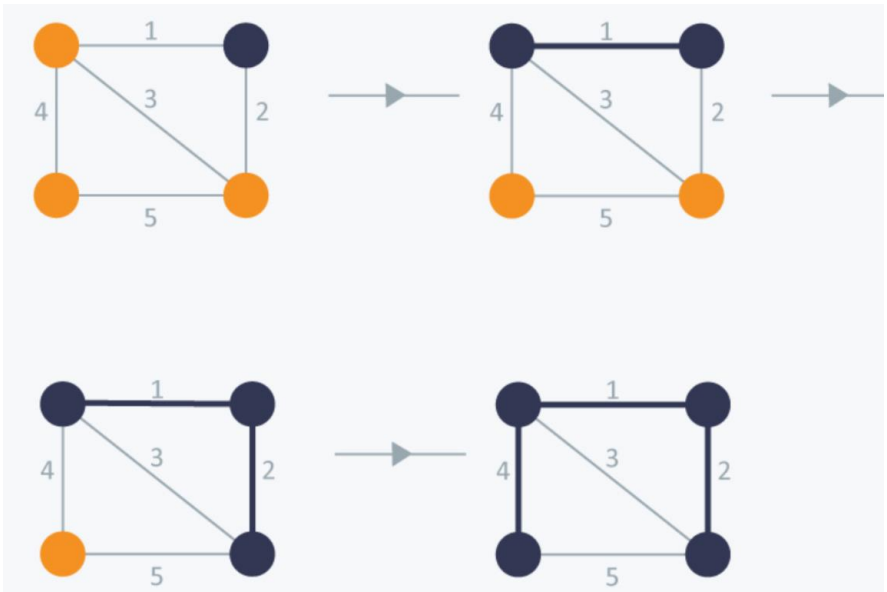
วิธีการของ Prim’s algorithm คือทำการเพิ่มจุดเข้าไปใน spanning tree ตามขั้นตอนดังนี้

- **แยกกลุ่มจุดยอดออกเป็น 2 ชุด:** ชุดแรกคือจุดยอดที่ถูกรวมเข้าไปในต้นไม้ที่กำลังเติบโตแล้ว และชุดที่สองคือจุดยอดที่ยังไม่ได้ถูกรวมเข้าไป
- **เลือกจุดยอดที่ "ระยะทางสั้นที่สุด”:** โดยต้องเป็นจุดที่เชื่อมต่อกับต้นไม้เดิม แต่ตัวมันเองยังไม่อยู่ในต้นไม้นั้น ขั้นตอนนี้สามารถทำได้โดยใช้ **Priority Queue (คิวลำดับความสำคัญ)** ด้วยการใส่จุดยอดที่เชื่อมกับต้นไม้หลักลงไปในคิว
- **ตรวจสอบการเกิดวงจร (Cycles):** ทำได้โดยการทำเครื่องหมาย (Mark) จุดที่ถูกเลือกไปแล้ว และจะใส่เฉพาะจุดที่ยังไม่ได้ทำเครื่องหมายลงใน Priority Queue เท่านั้น

<u>ตัวอย่าง</u> หา Minimum spanning tree โดยใช้ Prim’s algorithm



<u>ตัวอย่าง 5.3</u> หา Minimum spanning tree โดยใช้ Prim’s algorithm



ความรู้ : กราฟมีน้ำหนักติดลบ – Algorithm Floyd-Warshall น้ำหนักที่เส้นติดลบได้

1. ระบบสะสมแต้ม หรือ Cash Back (การเงิน)

สมมติว่าเรากำลังวางแผนเดินทางไปซื้อของในห้างสรรพสินค้า:

- **จุด A ไป B:** ต้องเสียค่าที่จอดรถ 40 บาท (Weight = 40)
- **จุด B ไป C:** ซื้อสินค้าชิ้นหนึ่งแล้วได้เงินคืน (Cash back) 100 บาท (Weight = -100)
- **จุด C ไป D:** เสียค่ารถเมล์ 20 บาท (Weight = 20)

ในการหา "เส้นทางที่ประหยัดที่สุด" ตัวเลขติดลบจะช่วยลดต้นทุนรวมลง ทำให้การไปแวะที่จุด B กลายเป็นตัวเลือกที่คุ้มค่ากว่าการเดินตรงจาก A ไป D

2. การจัดการพลังงานในรถยนต์ไฟฟ้า (EV)

- **ขับรถขึ้นเขา:** มอเตอร์ต้องใช้พลังงานจากแบตเตอรี่ (Weight เป็น **บวก** เช่น 10 kWh)
- **ขับรถลงเขา** (Regenerative Braking): ระบบปั่นไฟกลับเข้าแบตเตอรี่ ทำให้ได้พลังงานเพิ่มขึ้น (Weight เป็น **ลบ** เช่น -5 kWh)

หากต้องการหาเส้นทางที่ประหยัดแบตเตอรี่ที่สุด ต้องคำนวณว่าการยอมขับอ้อมไปทางที่มีเนินให้ลงเขา (ค่าติดลบ) อาจจะดีกว่าการขับทางราบที่เสียพลังงานคงที่

ข้อควรระวัง (เรื่องวงจรลบ) : จากตัวอย่างรถยนต์ไฟฟ้า:

- **เส้นติดลบ** (Negative edge)**:** คือการลงเขาแล้วได้ไฟคืน (สามารถคำนวณได้)
- **วงจรลบ:** คือการที่มีทางลงเขาที่ลงไปแล้ววนกลับมาจุดเดิมได้โดยที่ได้พลังงานมากกว่าที่เสียไปตอนขึ้น (เหมือนได้พลังงานฟรีไม่จำกัด ซึ่งในทางฟิสิกส์เป็นไปไม่ได้ และในทางอัลกอริทึมจะทำให้การคำนวณวนลูปไม่จบสิ้น)

6. Floyd-Warshall algorithm

https://www.programiz.com/dsa/floyd-warshall-algorithm

อัลกอริทึมฟลอยด์-วอร์ชอล (Floyd-Warshall Algorithm) คืออัลกอริทึมสำหรับการหาเส้นทางที่สั้นที่สุดระหว่างทุกคู่ของจุด (all pairs of vertices) ในกราฟที่มีการระบุน้ำหนัก (Weighted Graph) โดยอัลกอริทึมนี้มีความแตกต่างจากอัลกอรึทึมค้นหาเส้นทางที่สั้นที่สุดตัวอื่นๆ ตรงที่อัลกอรึทึมนี้จะใช้จุดแต่ละจุดในกราฟเป็น**จุดหมุน (pivot) หรือจุดผ่าน** เพื่อตรวจสอบว่าการเดินทางผ่านจุดดังกล่าวจะช่วยให้ได้เส้นทางที่สั้นที่สุดระหว่างจุดสองจุดใดๆ หรือไม่

อัลกอริทึมนี้ใช้หลักการ **กำหนดการพลวัต (Dynamic Programming)** เพื่อหาเส้นทางที่สั้นที่สุด (คำว่า **Dynamic Programming** หมายถึงการที่อัลกอริทึมแก้ปัญหาใหญ่ด้วยการนำคำตอบจากปัญหาย่อย ๆ มาใช้ต่อ ในที่นี้คือ อัลกอริทึมจะจำค่าระยะทางที่ดีที่สุดที่เคยหาได้ แล้วนำค่านั้นมาคำนวณต่อเรื่อย ๆ จนครบทุกจุด)

อัลกอริทึมนี้สามารถใช้งานได้กับทั้งกราฟแบบมีทิศทาง (Directed Graphs) และกราฟแบบไม่มีทิศทาง (Undirected Graphs) อย่างไรก็ตาม มันจะไม่สามารถทำงานได้กับกราฟที่มี **วงรอบติดลบ (Negative Cycles)** ซึ่งหมายถึงกรณีที่ผลรวมของค่าน้ำหนักของเส้นเชื่อมในวงรอบนั้น ๆ มีค่าเป็นลบ (เส้นทางที่เดินวนกลับมาที่เดิมแล้วผลรวมของค่าน้ำหนักลดลงเรื่อย ๆ ซึ่งทำให้หาทางที่สั้นที่สุดไม่ได้ เพราะยิ่งวนยิ่งค่าลดลงไปเรื่อย ๆ ไม่มีที่สิ้นสุด)

Floyd-Warshall algorithm จัดเป็นหนึ่งในวิธีการแก้ปัญหาการหาเส้นทางสั้นที่สุดแบบทุกคู่จุดยอด (All-pairs shortest path) และมักจะถูกแก้ไขโดยใช้การนำเสนอกราฟแบบ **เมทริกซ์ประชิด** (Adjacency Matrix)

หลักการณ์ของ Floyd-Warshall คือการถามคำถามเดิมซ้ำว่า:

"ถ้าเราเดินทางจากเมือง ***i*** ไปเมือง***j*** โดย **แวะพัก** ที่เมือง ***k*** แล้วจะใกล้กว่าเดินตรง***?"***

ถ้าการแวะพักทำให้ระยะทางสั้นลง เราจะอัปเดตข้อมูลระยะทางใหม่ทันที และเราจะทำแบบนี้กับ **ทุกจุดพัก** ที่เป็นไปได้

สูตรทางคณิตศาสตร์คือ:

```
dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

หมายความว่า ทำการทดสอบว่าการเดินไปที่จุด j โดยตรงจาก i" กับ "การแวะผ่านจุด k ก่อนแล้วค่อยไป j" ทางไหนมีระยะทางสั้นกว่ากัน

รายละเอียดสำคัญของอัลกอริทึม Floyd-Warshall:

- **หลักการทำงาน:** อัลกอริทึมจะค่อยๆ ปรับปรุงระยะทางที่สั้นที่สุดระหว่างคู่จุดยอด (I, j) โดยการพิจารณาจุดยอดตัวกลาง (k) ทีละจุด และเลือกเส้นทางที่สั้นกว่าระหว่างทางตรง หรือทางผ่านจุดตัวกลางนั้น
- **โครงสร้างข้อมูล:** ใช้เมทริกซ์ 2 มิติ (Adjacency Matrix) ในการเก็บระยะทางระหว่างคู่จุดยอด โดยจะมีการอัปเดตค่าในเมทริกซ์ไปเรื่อยๆ
- **ความซับซ้อนเชิงเวลา (Time Complexity):** มีความซับซ้อนอยู่ที่ $O(n^3)$ เมื่อ n คือจำนวนจุดยอด (Vertices)

  ข้อดี: เข้าใจและเขียนโค้ดได้ง่าย (Implement ง่าย) เหมาะสำหรับกราฟขนาดเล็กถึงปานกลาง

  ข้อจำกัด: ไม่สามารถทำงานได้หากกราฟมี [วงจรลบ (Negative Cycle)]

โดยสรุป Floyd-Warshall เป็นวิธีที่มีประสิทธิภาพในการหาเส้นทางที่สั้นที่สุดระหว่างทุกคู่จุดยอดในกราฟหนึ่งครั้ง โดยใช้แนวคิดการแบ่งปัญหาใหญ่เป็นปัญหาย่อยๆ

อัลกอรึทึม

ขั้นตอนที่ 1 – สร้างเมทริกซ์ประชิด (Adjacency Matrix) A โดยระบุน้ำหนัก (Cost) ของเส้นทั้งหมดที่มีอยู่ในกราฟ หากไม่มีเส้นทางเชื่อมต่อระหว่างจุดยอดสองจุดใดๆ ให้แทนค่าด้วยค่าอนันต์ (∞)

ขั้นตอนที่ 2 – สร้างเมทริกซ์ประชิด A_1 ต่อจาก A โดยคงค่าในแถวที่ 1 และคอลัมน์ที่ 1 ของเมทริกซ์เดิมไว้ใน A_1 สำหรับค่าที่เหลือ (สมมติว่าเป็น $A_1[i,j]$) ให้พิจารณาว่า **ถ้า A_1 [i, j] > A[i, k] + A[k, j]** ให้แทนที่ค่า $A_1[i, j]$ ด้วยผลรวมของ A[i, k] + A[k, j] แต่ถ้าไม่เป็นไปตามเงื่อนไขนี้ ให้คงค่าเดิมไว้ โดยในขั้นตอนนี้กำหนดให้ k = 1 (คือการใช้จุดยอดแรกเป็นจุดหมุนหรือ Pivot)

ขั้นตอนที่ 3 – ทำซ้ำขั้นตอนที่ 2 สำหรับทุกจุดยอดที่เหลือในกราฟ โดยเปลี่ยนค่า k ไปตามจุดยอดที่เป็นจุดหมุนแต่ละจุด จนกระทั่งได้เมทริกซ์ชุดสุดท้ายออกมา

ขั้นตอนที่ 4 – เมทริกซ์ประชิดชุดสุดท้ายที่ได้นั้น คือคำตอบที่แสดงระยะทางของเส้นทางที่สั้นที่สุดระหว่างทุกคู่จุดยอดในกราฟ

Pseudocode

```
Floyd-Warshall(w, n){ // w: weights, n: number of vertices
   for i = 1 to n do // initialize, D (0) = [wij]
      for j = 1 to n do{
         d[i, j] = w[i, j];
      }
      for k = 1 to n do // Compute D (k) from D (k-1)
         for i = 1 to n do
            for j = 1 to n do
               if (d[i, k] + d[k, j] < d[i, j]){
                  d[i, j] = d[i, k] + d[k, j];
               }
      return d[1..n, 1..n];
}
```

From the pseudocode above, the Floyd-Warshall algorithm operates using three for loops to find the shortest distance between all pairs of vertices within a graph. Therefore, the **time complexity** of the Floyd-Warshall algorithm is $O(n^3)$

<u>อธิบายเพิ่มเติม</u> เหตุผลที่เราต้องทำซ้ำให้ครบทุกโหนด (k=1 ไปจนถึง n) เพราะหัวใจของ Floyd-Warshall คือการ **"เปิดโอกาสให้ทุกจุดยอดได้ทำหน้าที่เป็นทางลัด"**

ถ้าเราทำไม่ครบ เราจะไม่ได้เส้นทางที่สั้นที่สุดจริง ๆ (Optimal Solution) ด้วยเหตุผลหลักๆ ดังนี้:

1. เพื่อค้นหา "จุดแวะพัก" ที่ซ่อนอยู่

จินตนาการว่าเราจะเดินทางจาก **เมือง** A ไป **เมือง** Z:

- รอบที่ i=1: อัลกอริทึมถามว่า "ถ้าแวะผ่านเมือง 1 จะสั้นลงไหม?"
- รอบที่ i=2: อัลกอริทึมถามว่า "ถ้าแวะผ่านเมือง 2 จะสั้นลงไหม?"

ถ้าเส้นทางที่สั้นที่สุดจริง ๆ คือการต้องผ่านทั้งเมือง 1, 3 และ 5 อัลกอริทึมจะค่อยๆ สร้างเส้นทางนี้ขึ้นมาจากการทำซ้ำแต่ละรอบ

2. การสร้างเส้นทางแบบ "ลูกโซ่" (Chain Reaction) นี่คือการทำ Dynamic Programming:

- ในรอบที่ i=1: เราอาจพบว่า A → 1 →B สั้นกว่า A → B
- ในรอบที่ i=2: เราอาจพบว่า B →2 → C สั้นกว่า B →C
- **เมื่อทำถึงรอบหลังๆ:** อัลกอริทึมจะสามารถรวมเส้นทางที่พบก่อนหน้าเข้าด้วยกัน เช่น กลายเป็น A → 1 → B → 2 →C

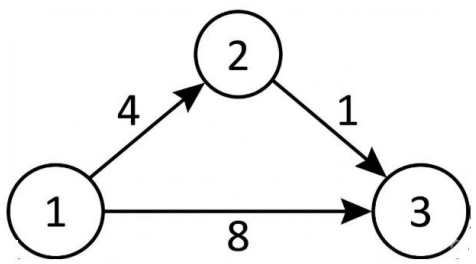
ถ้าเราหยุดทำกลางคัน: เราอาจจะรู้แค่ทางลัดจาก A ไป B แต่จะไม่รู้เลยว่าทางนั้นสามารถเชื่อมต่อไปยังจุดอื่นๆ เพื่อสร้างเส้นทางที่ยาวกว่าแต่ใช้ต้นทุนต่ำกว่าได้

3. กฎของความสมบูรณ์ (Completeness)

เพื่อให้มั่นใจว่าเราพิจารณา **"ทุกความเป็นไปได้"** ของการเดินทางระหว่างจุดสองจุด อัลกอริทึมจำเป็นต้องทดสอบทุกโหนดในกราฟให้ครบ เพราะเราไม่รู้ล่วงหน้าว่าโหนดไหนจะเป็น "ตัวเปลี่ยน" ที่ทำให้เส้นทางสั้นลงได้มากที่สุด

<u>ตัวอย่าง 6.1</u> สมมติเรามีกราฟที่มี 3 จุดยอด (Nodes): 1, 2, และ 3

ข้อมูลเส้นทาง (Edges):



- 1 → 2 : ระยะทาง **4**
- 2 → 3 : ระยะทาง **1**
- 1 → 3 : ระยะทาง **8** (ทางตรงแต่ไกล)
- ไม่มีเส้นทางย้อนกลับ (ค่าเป็น ∞)
- ระยะทางเข้าหาตัวเองเป็น **0**

เราจะเก็บข้อมูลในรูปแบบ ตาราง (Matrix) ขนาด 3 x 3

Step 0: เริ่มต้น (Initial State)

ใส่ค่าตามเส้นทางที่มีอยู่จริง ถ้าไม่มีเส้นเชื่อมถึงกันให้ใส่ ∞ (Infinity)

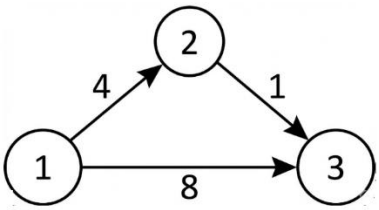
จาก \ ไป	1	2	3
1	0	4	8
2	∞	0	1
3	∞	∞	0

Step 1: ลองแวะพักที่จุด 1 (k=1)

เราจะตรวจสอบว่า "ถ้าเดินทางผ่านจุด 1 จะสั้นลงไหม?"

วิธีคิด

จาก \ ไป	1	2	3
1	0	4	8
2	∞	0	1
3	∞	∞	0



กรณีที่ต้องผ่านโหนด 1 มีทั้งหมด 2 กรณี ดังนี้

2 → 1 → 3 x เป็นไปไม่ได้

3 → 1 → 2 x เป็นไปไม่ได้

- เนื่องจากจุด 1 คือจุดเริ่มต้นของแถวแรก และจุดปลายของคอลัมน์แรก การเปลี่ยนแปลงมักจะไม่เกิดขึ้นในรอบนี้
- *ผลลัพธ์:* ตารางยังคงเดิม

Step 2: ลองแวะพักที่จุด 2 (k=2)

วิธีคิด

จาก \ ไป	1	2	3
1	0	4	8
2	∞	0	1
3	∞	∞	0

กรณีที่ต้องผ่านโหนด 2 มีทั้งหมด 2 กรณี ดังนี้

1 → 2 → 3 = 4 + 1 = 5

3 → 2 → 1 x เป็นไปไม่ได้

เราจะตรวจสอบทุกคู่ว่าถ้าผ่านจุด 2 จะดีกว่าไหม โดยเฉพาะคู่ **1 ไป 3**:

- **ทางเดิม (Direct):** 1 → 3 ใช้ระยะทาง **8**

- **ทางใหม่ (Via 2):** (1 → 2) + (2 → 3) = 4 + 1 = **5**

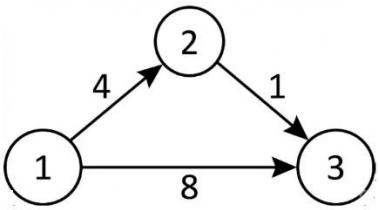
สรุป: 5 < 8 ดังนั้นเราเจอทางที่สั้นกว่าแล้ว เราจึงแก้เลข 8 เป็น 5

จาก \ ไป	1	2	3
1	0	4	5 (update)
2	∞	0	1
3	∞	∞	0

Step 3: ลองแวะพักที่จุด 3 (k=3)

วิธีคิด

จาก \ ไป	1	2	3
1	0	4	5
2	∞	0	1
3	∞	∞	0



กรณีที่ต้องผ่านโหนด 3 มีทั้งหมด 2 กรณี ดังนี้

1 → 3 → 2 x เป็นไปไม่ได้

2 → 3 → 1 x เป็นไปไม่ได้

ตรวจสอบว่าถ้าผ่านจุด 3 จะช่วยให้คู่ไหนสั้นลงไหม

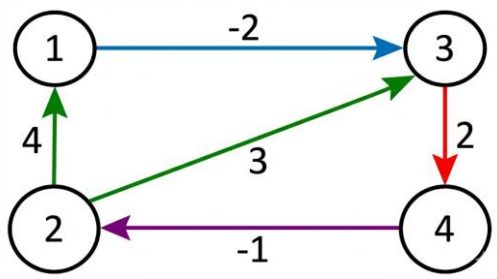
- ในกราฟนี้ จุด 3 เป็นจุดปลายทาง ไม่สามารถไปต่อที่ไหนได้ (∞) ดังนั้นการแวะที่ 3 จึงไม่ช่วยให้เดินทางไปที่อื่นได้เร็วขึ้น
- *ผลลัพธ์:* ตารางไม่เปลี่ยนแปลง

ผลลัพธ์สุดท้าย: ตารางสุดท้ายบอกเราว่า

- ทางที่สั้นที่สุดจาก **1 ไป 3** คือ **5** (คือ เส้น 1 → 2 → 3)

- ทางที่สั้นที่สุดจาก **1 ไป 2** คือ **4**
- ทางอื่น ๆ ที่เป็น ∞ คือไปไม่ถึง

<u>ตัวอย่าง 6.2</u> กราฟที่มี 4 จุดยอด (Nodes: 1, 2, 3, 4)



สมมติเรามีกราฟที่มีเส้นทางดังนี้:

1. **1 → 3**: น้ำหนัก -2 (มีค่าติดลบเพื่อให้เห็นความสามารถของอัลกอริทึม)
2. **2 → 1**: น้ำหนัก 4
3. **2 → 3**: น้ำหนัก 3
4. **3 → 4**: น้ำหนัก 2
5. **4 → 2**: น้ำหนัก -1

เป้าหมาย: หาทางสั้นที่สุดของทุกคู่

Step 0: สร้างตารางเริ่มต้น (Initial Matrix)

ตาราง D0 คือค่าเริ่มต้น ใส่ค่าตามเส้นทางที่มีจริง ถ้าไม่มีทางไป ใส่ ∞ (Infinity) และระยะทางไปยังตัวเองเป็น 0

D0	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

Step 1: ให้จุด 1 เป็นทางผ่าน (k=1)

วิธีคิด

D0	1	2	3	4
1	0	∞	-2	∞
2	4	0	3	∞
3	∞	∞	0	2
4	∞	-1	∞	0

กรณีที่ต้องผ่านโหนด 1 มีทั้งหมด 6 กรณี ดังนี้

2 → 1 → 3

2 → 1 → 4 x เป็นไปไม่ได้

3 → 1 → 2 x

3 → 1 → 4 x

4 → 1 → 2 x

4 → 1 → 3 x

คำถาม: ถ้าเดินผ่านโหนด 1 (i → 1 →j) ทางจะสั้นลงไหม?

สูตร: `D[i][j] = min(D[i][j], D[i][1] + D[1][j])`

ดูที่แถว 1 และคอลัมน์ 1 เป็นหลัก

- **พิจารณาคู่ 2 → 3:**
  - ทางเดิม (2 → 3): **3**
  - ทางใหม่ผ่าน 1 (2 → 1 → 3): 4 + (-2) = 2
  - **ผลลัพธ์:** 2 < 3 ดังนั้น **อัปเดตเป็น 2**

D1	1	2	3	4
1	0	∞	-2	∞
2	4	0	2 แก้	∞
3	∞	∞	0	2

D1	1	2	3	4
4	∞	-1	∞	0

Step 2: ให้จุด 2 เป็นทางผ่าน (k=2)

วิธีคิด

D1	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	∞	-1	∞	0

กรณีที่ต้องผ่านโหนด 2 มีทั้งหมด 6 กรณี ดังนี้

1 → 2 → 3 x เป็นไปไม่ได้

1 → 2 → 4 x เป็นไปไม่ได้

3 → 2 → 1 x เป็นไปไม่ได้

3 → 2 → 4 x เป็นไปไม่ได้

4 → 2 → 1, **-1 + 4 = 3**

4 → 2 → 3, 4 → 2 → 1 → 3, **-1 + 2 = 1**

สีเหลืองคือเอาค่าที่ update จาก step 1 มาใช้

คำถาม: ถ้าเดินผ่านโหนด 2 (i → 2 → j) ทางจะสั้นลงไหม?

- **พิจารณาคู่ 4 → 1:**
  - ทางเดิม: ∞
  - ทางใหม่ผ่าน 2 (4 → 2 → 1): (-1) + 4 = 3
  - **อัปเดตเป็น 3**
- **พิจารณาคู่ 4 → 3:**
  - ทางเดิม: ∞
  - ทางใหม่ผ่าน 2 (4 → 2 → 3): (-1) + 2 (เอาค่าล่าสุดจาก Step 1) = **1**
  - **อัปเดตเป็น 1**

D2	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	3 (แก้)	-1	1 (แก้)	0

Step 3: ให้จุด 3 เป็นทางผ่าน (k=3)

วิธีคิด

D2	1	2	3	4
1	0	∞	-2	∞
2	4	0	2	∞
3	∞	∞	0	2
4	3	-1	1	0

กรณีที่ต้องผ่านโหนด 3 มีทั้งหมด 6 กรณี ดังนี้

1 → 3 → 2 x เป็นไปไม่ได้

1 → 3 → 4 **-2+2** = 0

2 → 3 → 1 x เป็นไปไม่ได้

2 → 3 → 4 **2+2 = 4**

4 → 3 → 1 x เป็นไปไม่ได้

4 → 3 → 2 x เป็นไปไม่ได้

คำถาม: ถ้าเดินผ่านโหนด 3 (i → 3 → j) ทางจะสั้นลงไหม?

- **พิจารณาคู่ 1 → 4:**
  - ทางเดิม: ∞
  - ทางใหม่ผ่าน 3 (1 → 3 → 4): (-2) + 2 = 0
  - **อัปเดตเป็น 0**
- **พิจารณาคู่ 2 → 4:**

- ทางเดิม: ∞
- ทางใหม่ผ่าน 3 (2 → 3 → 4): 2 + 2 = 4
- **อัปเดตเป็น 4**

D3	1	2	3	4
1	0	∞	-2	0 (แก้)
2	4	0	2	4 (แก้)
3	∞	∞	0	2
4	3	-1	1	0

Step 4: ให้จุด 4 เป็นทางผ่าน (k=4)

วิธีคิด

D3	1	2	3	4
1	0	∞	-2	0
2	4	0	2	4
3	∞	∞	0	2
4	3	-1	1	0

กรณีที่ต้องผ่านโหนด 4 มีทั้งหมด 6 กรณี ดังนี้

1 → 4 → 2 0-1 =-1

1 → 4 → 3 0+1 = 1 x ไม่แก้

2 → 4 → 1 4+3 = 7 x ไม่แก้

2 → 4 → 3 4+1 = 5 x ไม่แก้

3 → 4 → 1 2+3 = 5

3 → 4 → 2 2-1 = 1

คำถาม: ถ้าเดินผ่านโหนด 4 (i → 4 → j) ทางจะสั้นลงไหม?

ตอนนี้เรามีข้อมูลเชื่อมโยงเกือบครบแล้ว ลองดูว่าจุด 4 เชื่อมใครได้อีก

- **พิจารณาคู่ 1 → 2:**
  - ทางเดิม: ∞
  - ทางใหม่ผ่าน 4 (1 → 4 → 2): 0 + (-1) = -1
  - **อัปเดตเป็น -1**
  - *(สังเกต: นี่คือเส้นทาง 1 → 3 → 4 → 2)*
- **พิจารณาคู่ 3 → 2:**
  - ทางเดิม: ∞
  - ทางใหม่ผ่าน 4 (3 → 4 → 2): 2 + (-1) = 1
  - **อัปเดตเป็น 1**

D4	1	2	3	4
1	0	-1 (แก้)	-2	0
2	4	0	2	4
3	5 (แก้)	1 (แก้)	0	2
4	3	-1	1	0

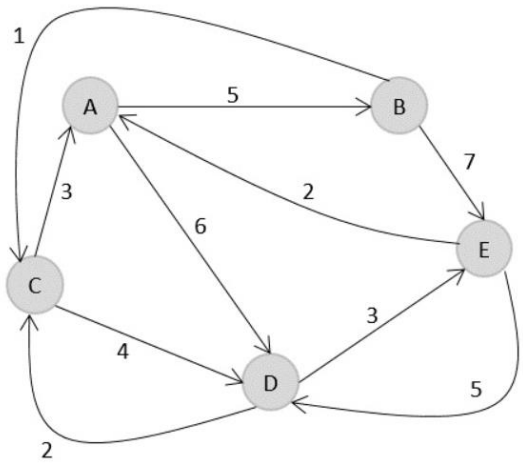
ตารางสุดท้าย (D4) คือคำตอบของระยะทางที่สั้นที่สุดสำหรับทุกคู่ เช่น:

- จาก **1 ไป 2**: สั้นที่สุดคือ **-1** (เส้นทาง: 1 → 3 → 4 → 2)
- จาก **2 ไป 4**: สั้นที่สุดคือ **4** (เส้นทาง: 2 → 1 → 3 → 4 หรือ 2 → 3 → 4 ซึ่งได้ค่าเท่ากันในตัวอย่างนี้ แต่จาก Step 1 เราเลือกเส้นทางที่อัปเดตแล้ว)
- จาก **3 ไป 1**: สั้นที่สุดคือ **4** (เส้นทาง: 3 → 4 → 2 → 1

ข้อสังเกต: อัลกอริทึมนี้ "ฉลาดขึ้น" เรื่อยๆ ในแต่ละ Step เพราะมันนำผลลัพธ์จากการคำนวณรอบก่อนหน้ามาใช้ (Dynamic Programming) ทำให้เจอเส้นทางที่ซับซ้อนอย่าง 1 → 3 → 4 → 2 ได้เอง

<u>ตัวอย่าง</u>

พิจารณากราฟระบุทิศทางแบบมีน้ำหนัก G = (V, E) ต่อไปนี้ จงหาเส้นทางที่สั้นที่สุดระหว่างจุดยอดทุกคู่ในกราฟ โดยใช้อัลกอริทึมของฟลอยด์-วอร์แชล (Floyd-Warshall algorithm)



Step 1: สร้างเมทริกซ์ประชิด (Adjacency Matrix) A โดยใช้ค่าระยะทางทั้งหมดเป็นค่าภายในเมทริกซ์	$A = \begin{matrix} 0 & 5 & \infty & 6 & \infty \\ \infty & 0 & 1 & \infty & 7 \\ 3 & \infty & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 2 & \infty & \infty & 5 & 0 \end{matrix}$
Step 2: ใช้เมทริกซ์ประชิด (Adjacency Matrix) ข้างต้นเป็นข้อมูลนำเข้า (Input) ให้สร้างเมทริกซ์ A1	

ขึ้นมาใหม่ โดยคงค่าในแถวแรกและหลักแรกเอาไว้ตามเดิม จากนั้นกำหนดให้ k=1 และแทนที่ค่าในตำแหน่งอื่นๆ ทั้งหมดด้วยผลรวมของ A[i,k] + A[k,j]	$A = \begin{matrix} 0 & 5 & \infty & 6 & \infty \\ \infty & & & & \\ 3 & & & & \\ \infty & & & & \\ 2 & & & & \end{matrix}$ $A_1 = \begin{matrix} 0 & 5 & \infty & 6 & \infty \\ \infty & 0 & 1 & \infty & 7 \\ 3 & 8 & 0 & 4 & \infty \\ \infty & \infty & 2 & 0 & 3 \\ 2 & 7 & \infty & 5 & 0 \end{matrix}$
Step 3: ……	$A_2 = \begin{matrix} & 5 & & & \\ \infty & 0 & 1 & \infty & 7 \\ & 8 & & & \\ & \infty & & & \\ & 7 & & & \end{matrix}$ $A_2 = \begin{matrix} 0 & 5 & 6 & 6 & 12 \\ \infty & 0 & 1 & \infty & 7 \\ 3 & 8 & 0 & 4 & 15 \\ \infty & \infty & 2 & 0 & 3 \\ 2 & 7 & 8 & 5 & 0 \end{matrix}$

Step 4:	$$A_3 = \begin{matrix} & & 6 & & \\ & & 1 & & \\ 3 & 8 & 0 & 4 & 15 \\ & & 2 & & \\ & & 8 & & \end{matrix}$$ $$A_3 = \begin{matrix} 0 & 5 & 6 & 6 & 12 \\ 4 & 0 & 1 & 5 & 7 \\ 3 & 8 & 0 & 4 & 15 \\ 5 & 10 & 2 & 0 & 3 \\ 2 & 7 & 8 & 5 & 0 \end{matrix}$$
Step 5:	$$A_4 = \begin{matrix} & & & 6 & \\ & & & 5 & \\ & & & 4 & \\ 5 & 10 & 2 & 0 & 3 \\ & & & 5 & \end{matrix}$$ $$A_4 = \begin{matrix} 0 & 5 & 6 & 6 & 9 \\ 4 & 0 & 1 & 5 & 7 \\ 3 & 8 & 0 & 4 & 7 \\ 5 & 10 & 2 & 0 & 3 \\ 2 & 7 & 7 & 5 & 0 \end{matrix}$$
Step 6:	$$A_5 = \begin{matrix} & & & & 9 \\ & & & & 7 \\ & & & & 7 \\ & & & & 3 \\ 2 & 7 & 7 & 5 & 0 \end{matrix}$$ $$A_5 = \begin{matrix} 0 & 5 & 6 & 6 & 9 \\ 4 & 0 & 1 & 5 & 7 \\ 3 & 8 & 0 & 4 & 7 \\ 5 & 10 & 2 & 0 & 3 \\ 2 & 7 & 7 & 5 & 0 \end{matrix}$$

สรุปความแตกต่าง Floyd-Warshall กับอัลกอริทึมอื่น

- **Dijkstra:** หาจาก *จุดเริ่มต้นจุดเดียว* ไปยังจุดอื่นทั้งหมด
- **Floyd-Warshall:** หาจาก *ทุกจุด* ไปยัง *ทุกจุด* (All-pairs shortest path) ในครั้งเดียว

Floyd-Warshall Algorithm in Python

```
V = 4
INF = float('inf')

def floydWarshall(graph):
    dist = list(map(lambda i: list(map(lambda j: j, i)), graph))

    for k in range(V):
        for i in range(V):
            for j in range(V):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    printSolution(dist)

def printSolution(dist):
    print("Shortest distances between every pair of vertices:")
    for i in range(V):
        for j in range(V):
            if dist[i][j] == INF:
                print("INF", end=" ")
            else:
                print(dist[i][j], end=" ")
        print()

if __name__ == "__main__":
    graph = [
        [0, 3, INF, 7],
        [INF, 0, 1, INF],
        [INF, INF, 0, 2],
        [INF, 5, INF, 0]
    ]
    floydWarshall(graph)
```

7. Graph Applications

โครงสร้างข้อมูลกราฟถูกนำไปใช้งานในหลากหลายรูปแบบ เช่น

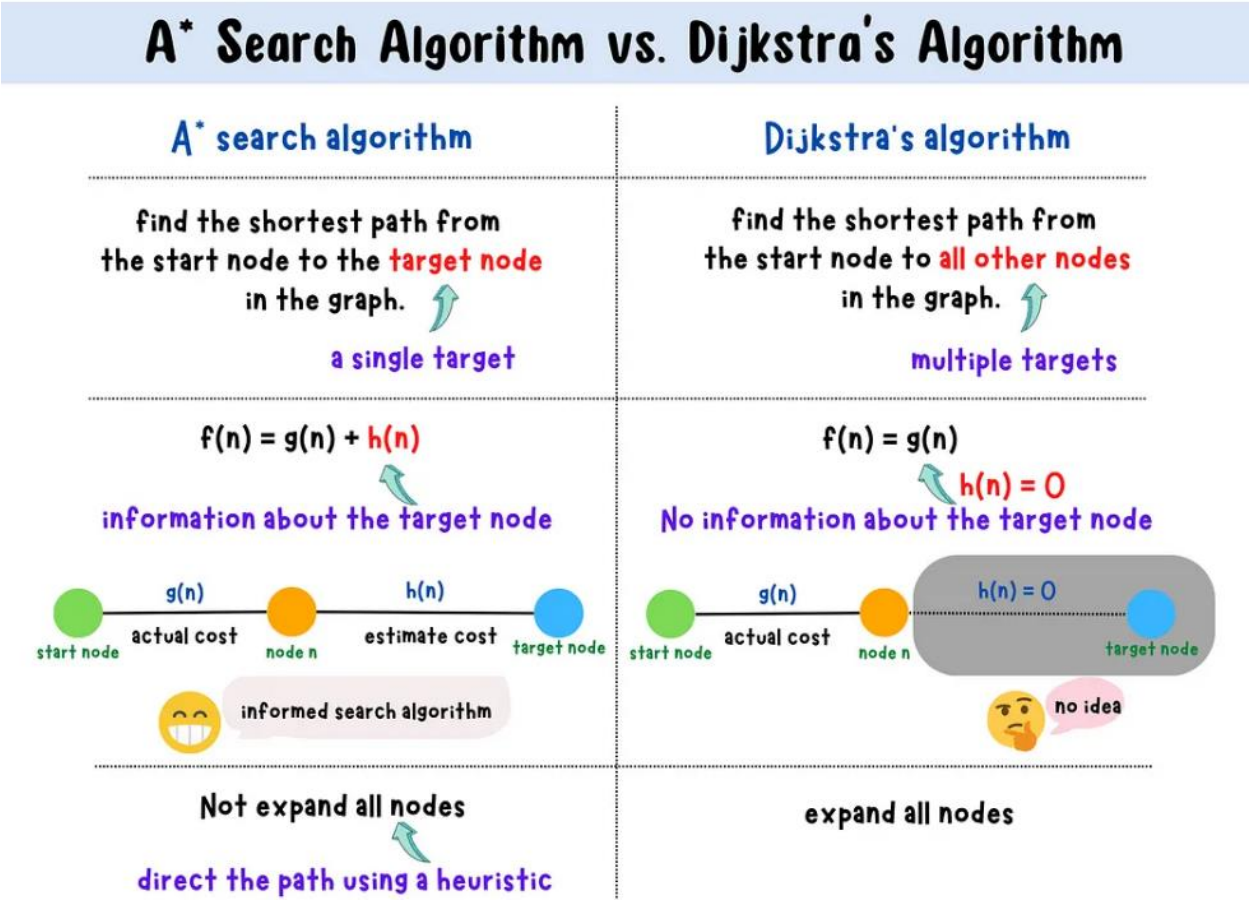
7.1 Google map, Flight networks (route optimizations)

ใน Google Maps จุดตัดของถนนตั้งแต่สองสายขึ้นไปจะถือว่าเป็น Node (โหนด) ในขณะที่ถนนที่เชื่อมต่อระหว่างสองโหนดจะเรียกว่า Edge (เส้นเชื่อม) โดย Google Maps จะใช้โครงสร้างข้อมูลแบบกราฟเพื่อคำนวณหาระยะทางที่สั้นที่สุดระหว่างจุดยอด (Vertices) สองจุด

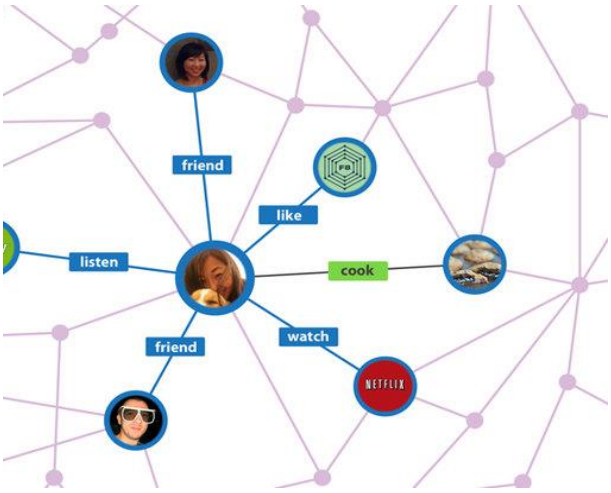
โดยพื้นฐานแล้ว Google Maps จะใช้อัลกอริทึมเกี่ยวกับกราฟอยู่ 2 รูปแบบ คือ Dijkstra’s algorithm และ *A** ***algorithm***, เพื่อคำนวณหาเส้นทางที่สั้นที่สุดจากจุด A (ต้นทาง) ไปยังจุด B (ปลายทาง)

*A** algorithm คืออัลกอริทึมที่ใช้ในการ **ค้นหาเส้นทาง (Pathfinding)** และการท่องไปในกราฟที่ได้รับความนิยมมากที่สุดตัวหนึ่ง โดยเฉพาะในการพัฒนาเกมและระบบนำทาง (GPS) อัลกอริทึมนี้ความสามารถในการหาเส้นทางที่ สั้นที่สุด และ มีประสิทธิภาพที่สุด โดยการผสมผสานข้อดีของ Dijkstra's Algorithm (ที่เน้นระยะทางจริง) กับ Greedy Best-First Search (ที่เน้นการคาดเดาความใกล้ไกล) เข้าด้วยกัน

Ref : https://yuminlee2.medium.com/a-search-algorithm-42c1a13fcf9f



7.2 Social network เช่น Facebook and Twitter



จุด/โหนด: Users, Pages, Places, Groups, Comments, Photos, Photo Albums, Stories, Videos, Notes, Events and so forth. สิ่งใดก็ตามที่มีคุณสมบัติในการจัดเก็บข้อมูลจะถือว่าเป็น จุด (Vertex)

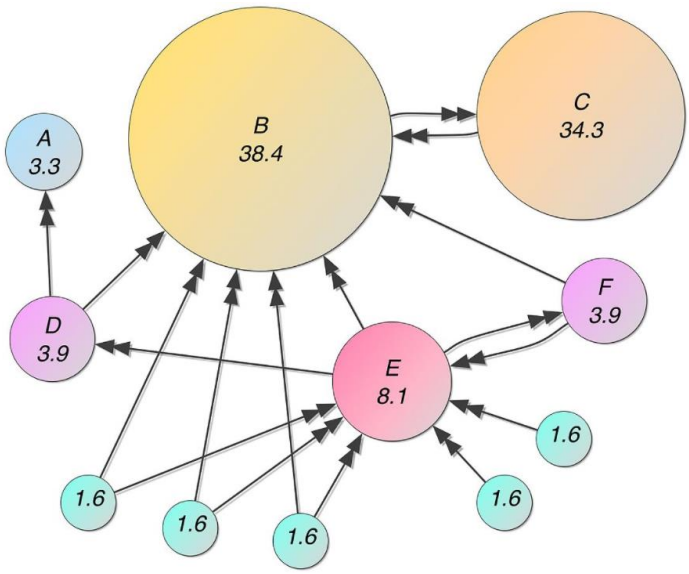
เส้น : ผู้ใช้ที่โพสต์รูปภาพ วิดีโอ หรือแสดงความคิดเห็น ผู้ใช้ที่อัปเดตข้อมูลสถานที่เกิดในโปรไฟล์ของตนเอง ผู้ใช้ที่มีการระบุสถานะความสัมพันธ์ หรือ ผู้ใช้ที่มีการลิงก์ไปยังรูปภาพของเพื่อน (User positing a photo, Video, or Comment, etc.; a user updating their profile with their place of birth; a relationship status user; a user linking a Friends' photo, etc.)

7.3 PageRank (PP)

PageRank คืออัลกอริทึมที่ Google Search ใช้ในการจัดอันดับหน้าเว็บต่าง ๆ ในหน้าแสดงผลลัพธ์การค้นหาของเครื่องมือค้นหา (Search Engine)

หลักการทำงานของ PageRank คือการนับจำนวนและประเมินคุณภาพของลิงก์ที่เชื่อมโยงไปยังหน้าเว็บนั้น ๆ เพื่อใช้คาดการณ์ความสำคัญของเว็บไซต์ในเบื้องต้น โดยมีสมมติฐานพื้นฐานที่ว่าเว็บไซต์ที่มีความสำคัญมากกว่า มักจะได้รับการลิงก์มาจากเว็บไซต์อื่น ๆ มากกว่านั่นเอง

- **หัวใจหลัก:** มองว่า "ลิงก์" คือ "คะแนนโหวต"
- **ปริมาณ:** ยิ่งคนลิงก์มาหามากเพจนั้นมาก เพจนั้นยิ่งได้คะแนนสูง
- **คุณภาพ:** ลิงก์จากเว็บที่น่าเชื่อถือ (เช่น เว็บมหาวิทยาลัยหรือเว็บข่าวระดับโลก) จะมีน้ำหนักมากกว่าลิงก์จากเว็บที่ไม่ (ค่อย) มีคนรู้จัก
- **สมมติฐาน:** เว็บที่ดี ใครๆ ก็อยากอ้างถึง



ภาพ PageRank

Ref: PageRank - Wikipedia

ในเชิงคณิตศาสตร์ ค่า PageRank สำหรับเครือข่ายพื้นฐานจะแสดงผลเป็นเปอร์เซ็นต์ (ขณะที่ Google จะใช้มาตรวัดแบบลอการิทึม) จากตัวอย่างนี้ Page C มีค่า PageRank สูงกว่า Page E แม้ว่าจะมีจำนวนลิงก์ที่เชื่อมไปยัง Pae C น้อยกว่าก็ตาม ทั้งนี้เป็นเพราะลิงก์เพียงหนึ่งเดียวที่เชื่อมไปยัง Page C นั้นมาจากหน้าที่มีความสำคัญสูง (คือ Page B) จึงทำให้ลิงก์นั้นมีมูลค่าสูงตามไปด้วย Page Rank simulator https://tools.withcode.uk/pagerank/

อ่านเพิ่มเติม PageRank, Feb 2026

ปัจจุบัน Google ไม่ได้ใช้แค่ PageRank อย่างเดียวในการจัดอันดับผลการค้นหา แต่ในระบบการจัดอันดับของ Google ในปี ค.ศ. 2026 เป็นโครงข่ายที่ซับซ้อนมาก ซึ่งรวมเอาปัจจัยต่าง ๆ กว่าหลายร้อยรายการ (Signals) มาประมวลผลร่วมกัน โดย PageRank เป็นเพียงหนึ่งใน "รากฐาน" ที่ยังคงใช้อยู่ แต่ถูกปรับปรุงและเสริมด้วยระบบอื่น ๆ ดังนี้:

1. PageRank ยังอยู่ แต่ "ซ่อนอยู่หลังม่าน"

- Google ยังคงใช้ระบบที่วิเคราะห์โครงสร้างลิงก์ (Backlinks) เพื่อประเมินความน่าเชื่อถือ แต่อัลกอริทึม PageRank แบบเดิมได้ถูกพัฒนาต่อยอดไปมาก
- **จุดที่เปลี่ยนไป:** Google ไม่เปิดเผยคะแนน PageRank ให้คนทั่วไปเห็นมานานแล้ว (Toolbar PageRank ถูกยกเลิกไป) และปัจจุบันเน้นที่ "คุณภาพและความเกี่ยวข้อง" ของลิงก์มากกว่าแค่ "จำนวน"

2. ยุคของ AI และการเข้าใจความหมาย (Deep Learning)

ปัจจุบัน Google ใช้ AI หลายตัวมาทำงานร่วมกันเพื่อให้เข้าใจว่าเรา "ต้องการหาอะไร" กันแน่:

- **RankBrain:** ใช้ AI ช่วยทำความเข้าใจเจตนาของผู้ใช้ (User Intent)

- **BERT:** ช่วยให้ Google เข้าใจความหมายของประโยคที่ซับซ้อน หรือคำที่มีความหมายหลายแง่มุม (Contextual Understanding)
- **Gemini:** ในปี 2026 นี้ Google ได้นำโมเดล Gemini เข้ามาเป็นหัวใจหลักในการสร้าง **AI Overviews (SGE)** เพื่อสรุปคำตอบให้ผู้ใช้ทันทีโดยไม่ต้องคลิกเข้าเว็บด้วยซ้ำ

3. ปัจจัยเรื่อง "ประสบการณ์และความเชื่อถือ" (E-E-A-T)

นี่คือเกณฑ์ที่ Google ให้ความสำคัญมากที่สุดในปีนี้:

- **Experience (ประสบการณ์):** เนื้อหาต้องแสดงให้เห็นว่าผู้เขียนได้ใช้งานหรือมีประสบการณ์จริง
- **Expertise (ความเชี่ยวชาญ):** ผู้เขียนมีความรู้ลึกซึ้งในเรื่องนั้น ๆ
- **Authoritativeness (การมีอิทธิพล):** เว็บไซต์เป็นที่ยอมรับในแวดวงนั้น
- **Trustworthiness (ความน่าเชื่อถือ):** ข้อมูลมีความถูกต้อง โปร่งใส และปลอดภัย

4. ประสบการณ์การใช้งาน (Page Experience)

Google ยังตรวจวัดความเร็วของหน้าเว็บ (Core Web Vitals) การรองรับมือถือ (Mobile-friendliness) และความปลอดภัย (HTTPS) หากเว็บดีแต่โหลดช้ามาก อันดับก็ร่วงลงได้

สรุป ถ้าเปรียบเทียบเป็น **"การสมัครงาน"**

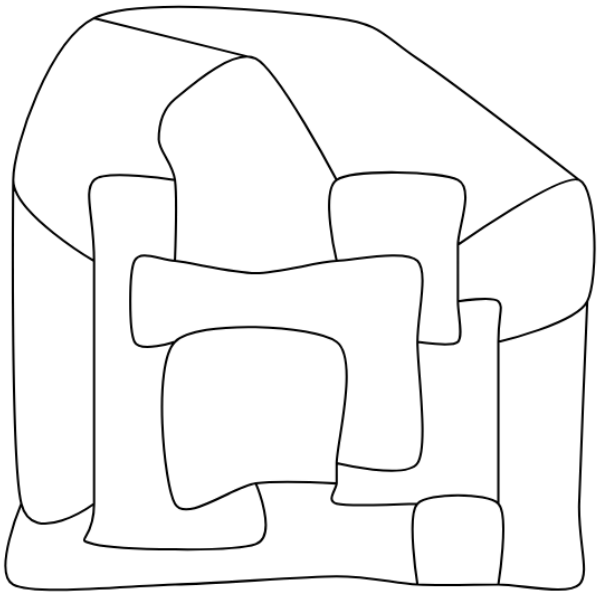
- **PageRank** = ใบปริญญาและการรับรองจากคนอื่น (พิจารณาว่าใครอ้างถึงเราบ้าง)
- **AI Algorithms** = การสัมภาษณ์ (คุยกันรู้เรื่องไหม เข้าใจคำถามหรือเปล่า)
- **E-E-A-T** = ผลงานจริงและประวัติการทำงาน (เราเก่งจริงและไว้ใจได้แค่ไหน)

7.4 Graph coloring (การระบายสีกราฟ)

การระบายสีกราฟ คือการ ให้สีกับจุดยอด (Vertices) ของกราฟ โดยมีเงื่อนไขเหล็กว่า **จุดยอดสองจุดใดๆ ที่มีเส้นเชื่อม (Edge) ถึงกัน ห้ามมีสีเดียวกัน** เป้าหมายสูงสุดคือการใช้จำนวนสีให้น้อยที่สุดเท่าที่จะเป็นไปได้ ซึ่งจำนวนสีที่น้อยที่สุดนี้เรียกว่า **เลขโครมาติก (Chromatic Number)** แทนด้วยสัญลักษณ์ $\chi(G)$

<u>ตัวอย่าง</u>

นักทำแผนที่ในดินแดนสมมติ ได้วาดเส้นแบ่งเขตการปกครองทั่วทั้งดินแดนเอาไว้ดังภาพ เพื่อให้แผนที่นั้นดูสวยงาม พวกเขาจึงต้องการระบายสีให้กับแต่ละพื้นที่ โดยมีเงื่อนไขว่า **เขตการปกครองที่มีพรมแดนติดกันต้องระบายด้วยสีที่ต่างกัน** แต่พื้นที่ที่อยู่ห่างไกลกันสามารถใช้สีเดียวกันได้ จำนวนสีที่น้อยที่สุดที่นักทำแผนที่สามารถใช้เพื่อให้บรรลุภารกิจนี้คือเท่าใด?



"ทฤษฎีสี่สี" (Four Color Theorem) ซึ่งระบุว่า:

สำหรับแผนที่ใด ๆ บนระนาบ (เหมือนแผนที่บนกระดาษ) เราจะใช้สี **ไม่เกิน 4 สี** เสมอ เพื่อระบายให้พื้นที่ที่ติดกันมีสีต่างกัน

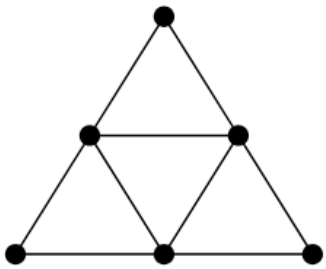
ตัวอย่าง ปัญหาที่ใช้การระบายสีกราฟ

- การจัดตารางเวลา หรือตารางสอน (Making a schedule or Time table) **จุดยอด"** และถ้ามีนักเรียนลงทะเบียนเรียนทั้งสองวิชาพร้อมกัน เราจะลาก **"เส้นเชื่อม"** ระหว่างสองวิชานั้น
    - **เงื่อนไข:** วิชาที่มีเส้นเชื่อมกัน (มีนักเรียนคนเดียวกันเรียน) จะสอบเวลาเดียวกันไม่ได้
    - **การลงสี:** สีแต่ละสีแทน "ช่วงเวลาสอบ"
    - **โจทย์:** เราต้องหาจำนวนช่วงเวลาสอบที่น้อยที่สุด เพื่อไม่ให้นักเรียนคนไหนต้องสอบซ้อนกัน

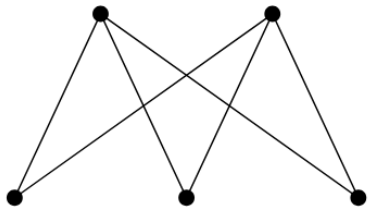
- การจัดสรรคลื่นความถี่วิทยุในเครือข่ายมือถือ (Mobile Radio Frequency Assignment)
- Akamai Experience the Edge บริษัท Akamai ดำเนินกิจการเครือข่ายที่มีเซิร์ฟเวอร์หลายพันเครื่อง เพื่อใช้ในการกระจายเนื้อหา (Content) บนอินเทอร์เน็ต พวกเขาต้องติดตั้งซอฟต์แวร์ใหม่หรืออัปเดตซอฟต์แวร์เดิมแทบจะทุกสัปดาห์ แต่ปัญหาคือ **ไม่สามารถอัปเดตเซิร์ฟเวอร์ทุกเครื่องพร้อมกันได้** เพราะอาจต้องปิดระบบชั่วคราวเพื่อติดตั้ง ในขณะเดียวกัน **ก็ไม่ควรใช้วิธีอัปเดตทีละเครื่อง** เพราะจะใช้เวลานานเกินไป นอกจากนี้ ยังมีเซิร์ฟเวอร์บางชุดที่ **ห้ามปิดระบบพร้อมกัน** เนื่องจากทำหน้าที่สำคัญในระบบ (Critical functions) นี่คือตัวอย่างการประยุกต์ใช้ปัญหาการระบายสีกราฟ (Graph Coloring) ในการจัดตารางเวลา ซึ่งผลปรากฏว่าการใช้ **8 สี** ก็เพียงพอที่จะระบายกราฟที่มีโหนดถึง 75,000 โหนดได้ นั่นหมายความว่า พวกเขาสามารถติดตั้งตัวอัปเดตให้เสร็จสิ้นได้ภายใน 8 รอบ (8 passes) เท่านั้น

<u>ตัวอย่าง</u> จงหาว่ากราฟแต่ละข้อต่อไปนี้ต้องใช้สีจำนวนน้อยที่สุดเท่าไหร่จึงจะทำให้พื้นที่ติดกันมีสีต่างกัน (หา $\chi(G)$)

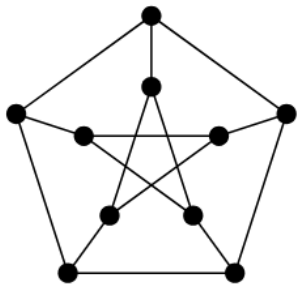
1.)



2.)



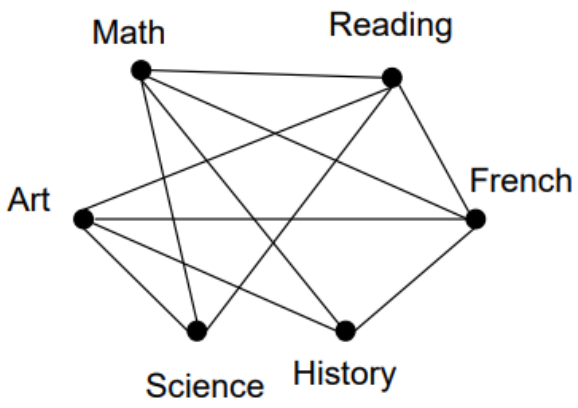
3.)



<u>ตัวอย่าง</u> ในการจัดตารางสอบปลายภาคสำหรับภาคฤดูร้อน มีการสอบที่แตกต่างกัน 6 วิชา ซึ่งต้องจัดให้กับนักเรียน 7 คน โดยตารางด้านล่างแสดงวิชาสอบที่นักเรียนแต่ละคนต้องเข้าสอบ จงหาว่าจะสามารถจัดสอบโดยใช้ช่วงเวลา (Time Slots) น้อยที่สุดได้กี่ช่วง

Exam	Minnie	JayB	Charles	Lisa	Eva	Jack	Adam
Math	X		X		X		X
Art		X		X		X	
Science	X	X					X
History			X			X	
French					X	X	
Reading	X	X		X	X		X

กราฟด้านล่างแสดงรายวิชาที่มีนักศึกษาอย่างน้อย 1 คนลงทะเบียนเรียน นั่นคือห้ามจัดสอบในเวลาเดียวกัน



หาเลขโครมาติก ($\chi(G)$)

ดังนั้น เราจึงต้องใช้ช่วงเวลาสอบ (Time Slots) ทั้งหมด 3 ช่วง เพื่อจัดตารางสอบ ตัวอย่างเช่น:

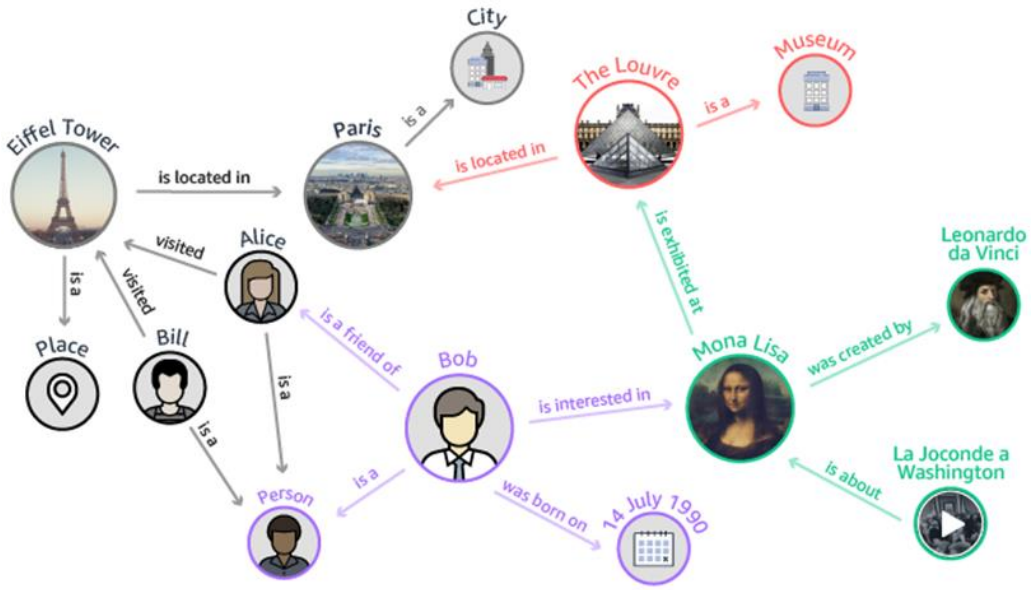
9.00 – 9.50	10.00 – 10.50	11.00 – 11.50
Math	Reading	French
Art	History	Science

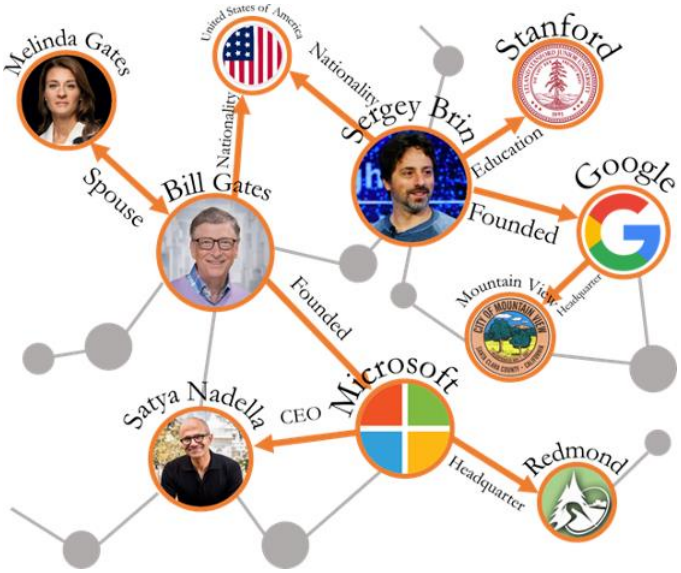
7.5 Knowledge graphs

กราฟความรู้ (Knowledge Graph) คือการจัดเก็บความหมายทางภาษา (Semantics) ของขอบเขตเนื้อหาเฉพาะทางใดทางหนึ่ง โดยใช้ชุดคำนิยามของ Concepts, คุณลักษณะของมโนทัศน์เหล่านั้น (Properties) ความสัมพันธ์ระหว่างกัน (Relations) รวมถึงข้อจำกัดทางตรรกะ (Logical constraints) ที่กำหนดไว้ว่าต้องเป็นจริง

โดยที่

- **Semantics (อรรถศาสตร์/ความหมาย):** คือการทำให้คอมพิวเตอร์ "เข้าใจ" ว่าข้อมูลนั้นคืออะไร ไม่ใช่แค่จำว่าเป็นตัวอักษรเท่านั้น
- **Concepts (มโนทัศน์):** เปรียบเสมือน **"ประเภทของจุดยอด"** เช่น ในวงการหนัง Concept คือ นักแสดง ผู้กำกับ หรือ ภาพยนตร์
- **Relations (ความสัมพันธ์):** เปรียบเสมือน **"เส้นเชื่อมที่มีป้ายกำกับ"** เช่น เส้นเชื่อมจากจุด A ไป B ไม่ได้บอกแค่ว่าติดกัน แต่บอกว่า "A **แสดงในเรื่อง** B"
- **Logical Constraints (ข้อจำกัดทางตรรกะ):** คือกฎของกราฟ เช่น "คนหนึ่งคนต้องมีวันเกิดเพียงวันเดียวเท่านั้น" หรือ "ถ้า A เป็นพ่อของ B และ B เป็นพ่อของ C แล้ว A ต้องเป็นปู่ของ C"





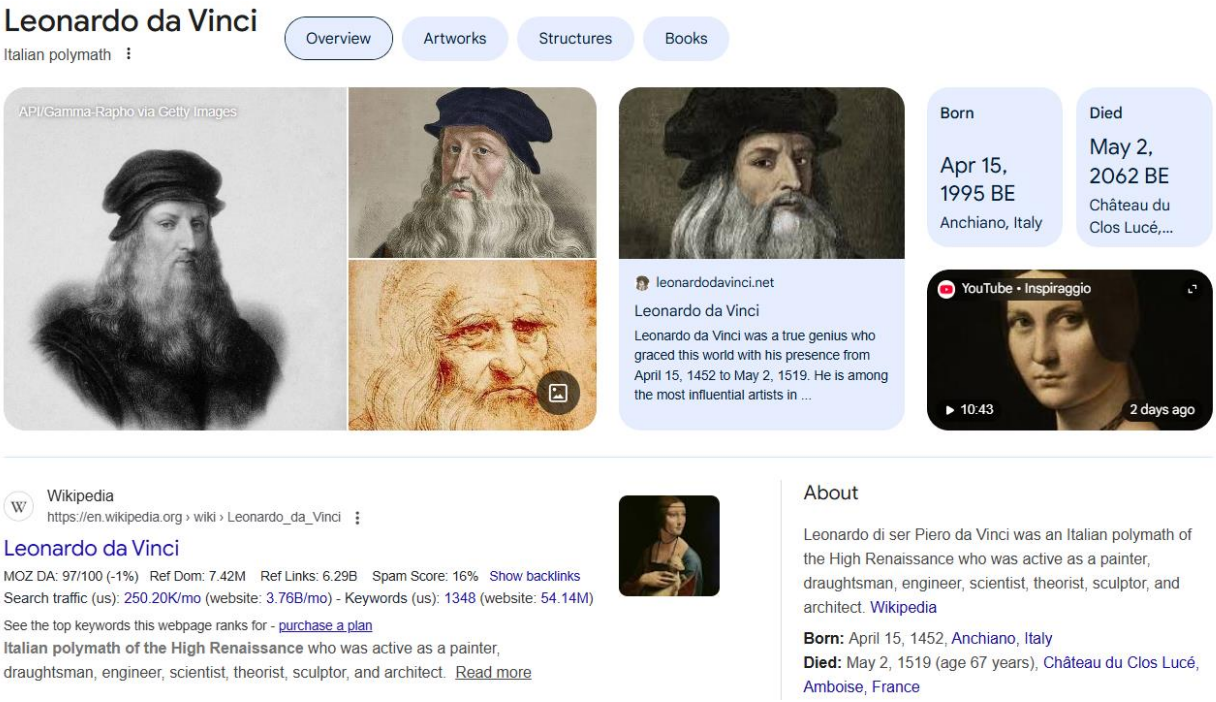
ภาพ Knowledge Graph Example of IT Company founder

(ที่มา https://medium.com/@fakrami/re-evaluation-of-knowledge-graph-completion-methods-7dfe2e981a77)

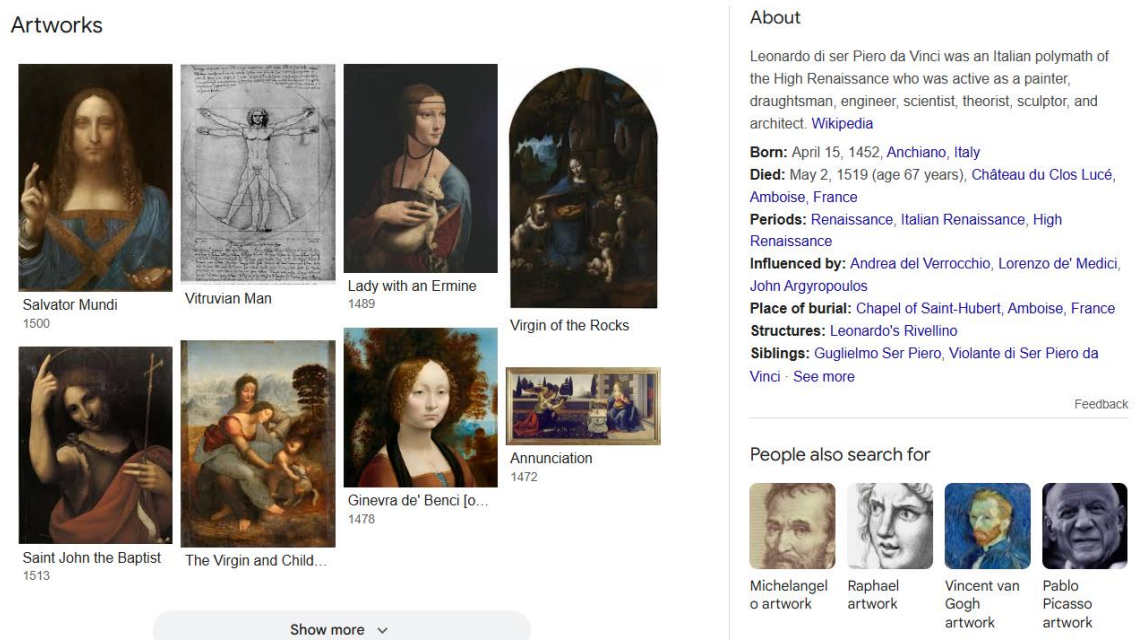
จากภาพข้างบน การแสดงความรู้ในรูปแบบของโครงสร้างประโยค เช่น *“Bill Gates founded Microsoft”* สามารถแสดงในรูปแบบของ Triples ได้เป็น (Bill Gates, founded,

Microsoft) เมื่อ Bill Gates คือตัวแทนข้อมูล (Instance data) ของ Subject entity ส่วน Microsoft คือตัวแทนข้อมูลของ Object entity และ founded เป็นตัวแทนข้อมูลของ Predicate ที่แสดงความสัมพันธ์ระหว่าง Bill Gates และ Microsoft

<u>ตัวอย่าง</u> เมื่อเราค้นหาคำว่า บน Google จะปรากฏข้อมูลดังนี้



เมื่อเลือก กด “Artworks” ปรากฏข้อมูลดังนี้



การลงสีกราฟ (Graph Coloring) vs. กราฟความรู้ (Knowledge Graph)

ขณะที่ **การลงสีกราฟ (Graph Coloring)** สนใจแค่ว่า "จุดติดกันห้ามซ้ำกัน" แต่ **กราฟความรู้ (Knowledge Graph)** จะเน้นไปที่การระบุว่า "จุดติดกันนั้นสัมพันธ์กันอย่างไร" เพื่อให้ AI (เช่น Google Search หรือ Siri) สามารถตอบคำถามที่ซับซ้อนได้

7.6 การแลกเปลี่ยนเงินตรา ใช้อัลกอรึทึม Floyd-Warshall

❖ สมมุติเราอยู่ในตลาดแลกเปลี่ยนเงินตราที่มีบูธแลกเงินหลายสกุล โดยมีอัตราแลกเปลี่ยนที่ "ไม่สมดุล" กันชั่วขณะดังนี้:

- **เส้นทางที่ 1:** เอาเงิน 100 ดอลลาร์ ไปแลกเป็น ยูโร ได้ 90 ยูโร
- **เส้นทางที่ 2:** เอา 90 ยูโร ไปแลกเป็น เงินเยน ได้ 15,000 เยน
- **เส้นทางที่ 3:** เอา 15,000 เยน กลับมาแลกเป็น ดอลลาร์ ได้ 110 ดอลลาร์

จะเห็นว่าถ้าเดินครบวงจร (ดอลลาร์ → ยูโร → เยน → ดอลลาร์) เราเริ่มจาก 100 ดอลลาร์ แต่จบที่ 110 ดอลลาร์ ดังนั้นเราได้กำไรมา 10 ดอลลาร์

❖ การใช้ "ค่าติดลบ" (The Log Trick)

ปกติเวลาเราคำนวณเงิน เราใช้การ **"คูณ"** อัตราแลกเปลี่ยนไปเรื่อยๆ แต่ Floyd-Warshal ออกแบบมาเพื่อการ **"บวก"** ระยะทางเข้าด้วยกัน นักคณิตศาสตร์จึงใช้สมบัติของ **Logarithm** มาช่วย เพื่อเปลี่ยนการคูณให้เป็นการบวก:

สูตรคือ: W = -log(Exchange Rate)

ทำไมต้องมีเครื่องหมายลบ (-) ?

- ถ้าอัตราแลกเปลี่ยนดีมาก (แลกแล้วได้เงินมากขึ้น) ค่า Log จะเป็นบวก พอใส่เครื่องหมายลบข้างหน้า ค่าน้ำหนักในกราฟ (Weight) จะกลายเป็น **"ติดลบ"**
- **ยิ่งกำไรเยอะ ค่าก็ยิ่งติดลบมาก**

❖ เชื่อมโยงกับ Floyd-Warshall

เมื่อเราแปลงอัตราแลกเปลี่ยนทั้งหมดในตลาดให้เป็นค่า **Negative Weight** (น้ำหนักติดลบ) แล้ว:

1. **การหาเส้นทางที่สั้นที่สุด (Shortest Path):** ในที่นี้จะหมายถึงการหาเส้นทางที่ **"ติดลบมากที่สุด"** หรือก็คือเส้นทางที่ได้ **"กำไรมากที่สุด"** นั่นเอง
2. **ถ้าเจอ "วงจรลบ" (Negative Cycle):** ในโลกของการเงิน คือขุมทรัพย์ เพราะหมายความว่าเราสามารถเดินวนแลกเงินในวงจรนั้นได้เรื่อย ๆ และเงินของเราจะเพิ่มขึ้นมหาศาลอย่างไม่สิ้นสุด (ในโลกความจริง ช่องว่างนี้จะหายไปในเสี้ยววินาที เพราะมีคนทำตาม หรือ Robot ของธนาคารเข้ามาจัดการ)

สรุป

- **จุด (Node):** สกุลเงินต่าง ๆ (USD, EUR, JPY)
- **ขอบติดลบ (Negative Edge):** การแลกเงินที่ให้ผลตอบแทนคุ้มค่ามาก
- **วงจรลบ (Negative Cycle):** วงจรการแลกเงินที่ทำให้คุณรวยขึ้นเรื่อย ๆ ทุกครั้งที่แลกจนครบวง

นี่คือเหตุผลที่ธนาคารหรือบริษัท Funding ต้องใช้การหา **Shortest Path** เพื่อตรวจจับช่องว่างราคาในตลาด

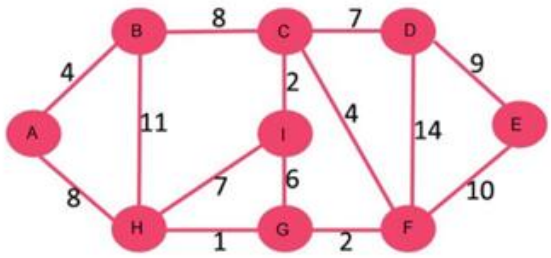
<u>แบบฝึกหัด</u>

1. จง**เติมค่าลงในตารางให้ถูกต้อง** โดยใช้ Dijkstra's algorithm เพื่อหาระยะทางที่สั้นที่สุดจากจุดเริ่มต้นที่กำหนดไปยังจุดอื่นในกราฟโดย โดยนักศึกษาแต่ละคนให้มีจุดเริ่มต้นที่แตกต่างกันดังนี้

จุดเริ่มต้น = (รหัสนักศึกษาตัวสุดท้าย) mod 8

เช่น รหัสนักศึกษา 67305657 นั่นคือ 7 mod 8 = 7 ดังนั้นจุดเริ่มต้นในกราฟคือ โหนด H

ผลลัพธ์ 0 จุดเริ่มต้น A	ผลลัพธ์ 1 จุดเริ่มต้น B	ผลลัพธ์ 2 จุดเริ่มต้น C	ผลลัพธ์ 3 จุดเริ่มต้น D
ผลลัพธ์ 4 จุดเริ่มต้น E	ผลลัพธ์ 5 จุดเริ่มต้น F	ผลลัพธ์ 6 จุดเริ่มต้น G	ผลลัพธ์ 7 จุดเริ่มต้น H



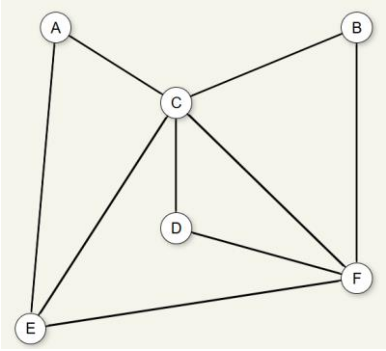
Visited = {

Unvisited = { A, B, C, D, E, F, G, H, I }

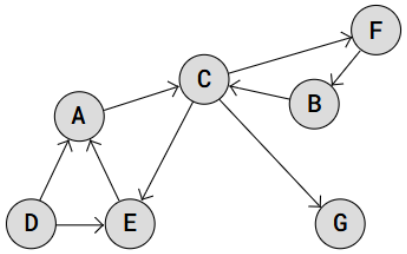
จุด	ระยะทางสั้นที่สุด จากจุด _____	จุดก่อนหน้า
A	0	
B		
C		
D		
E		
F		
G		
H		
I		

2. จากกราฟที่กำหนดให้ จงหาลำดับที่เกิดจากการทำทั้ง Breadth first search และ Depth first search กำหนดให้โหนด D เป็นจุดเริ่มต้น

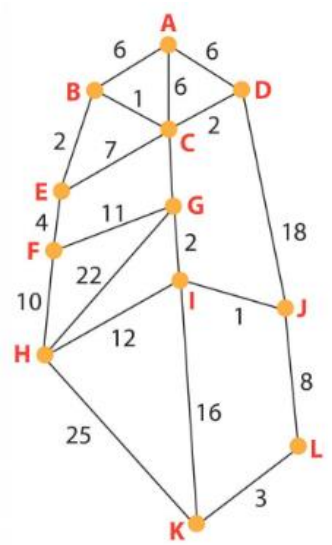
2.1)



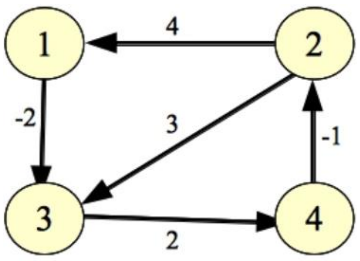
2.2)



3. กำหนดกราฟดังข้างล่าง จงหา Minimum spanning tree ด้วยวิธีการของ Kruskal’s algorithm และ Prim’s algorithm



4. กราฟประกอบด้วย 4 โหนด และน้ำหนักของเส้นในกราฟแสดงดังภาพข้างล่าง



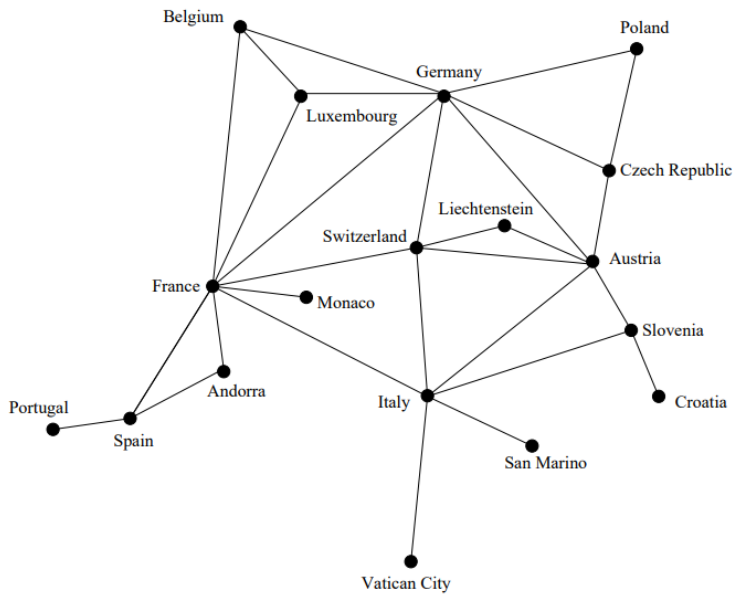
4.1) นำเสนอกราฟที่อยู่ในรูป Adjacency Matrix และ Adjacency List

4.2) จง**แสดงวิธี**การหาระยะทางที่สั้นที่สุดโดยใช้วิธีการของ Floyd-Warshall

5. เพื่อนกลุ่มหนึ่งจำนวน 10 คน ซึ่งประกอบด้วยผู้ชาย 5 คน และผู้หญิง 5 คน ได้ตัดสินใจเดินทางไปเที่ยวด้วยกัน แต่มีเพื่อนหลายคู่ในกลุ่มนี้เคยเป็นแฟนกันมาก่อนและความสัมพันธ์จบลงแล้ว ในการเดินทางครั้งนี้พวกเขาต้องแบ่งกลุ่มกันนั่งรถ โดยมีเงื่อนไขว่า

5.1). คนสองคนที่เคยหรือกำลังเป็นแฟนกันห้ามร่วมนั่งรถคันเดียวกัน หากความสัมพันธ์ที่เคยเกิดขึ้นทั้งหมดเป็นแบบ ชาย-หญิง (heterosexual) เท่านั้น

5.2). ทุกคนเคยเป็นแฟนกันหมด

ในแต่ละกรณีต้องใช้จำนวนรถที่น้อยที่สุดที่เท่าใด จงวาดสถานการณ์นี้ด้วยกราฟในแต่ละเงื่อนไขด้วย

6. จงหาว่าต้องใช้จำนวนสีน้อยที่สุดเท่าใด จึงจะทำให้ประเทศที่มีขอบติดกันเป็นคนละสี